

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Oscar Acosta

7 de abril de 2026

10:04

1. Entorno experimental

El la realización de esta tarea se utilizó un solo computador de las siguientes especificaciones:

- **Procesador:** AMD Ryzen 5 4500U con Radeon Graphics (2.38 GHz)
- **Memoria RAM:** 16 GB
- **Sistema Operativo:** Windows 11
- **Entorno de ejecución:** Subsistema de Windows para Linux (WSL) versión 2.6.3.0
- **Compilador:** GCC (g++)

Con respecto los datasets utilizados corresponden a los descritos en el enunciado de la tarea, pero es importante mencionar que para los algoritmos de ordenamiento no se ejecutó la totalidad de los casos más grandes, debido a que superaban las 4 horas de ejecución en Quick Sort, debido a que la tendencia de las curvas de rendimiento entorno a la memoria y tiempo ya eran claramente reconocibles con las dimensiones anteriores.

Finalmente, el tiempo de ejecución se cronometró con la librería <chrono>. Para evaluar la complejidad espacial y evadir la contaminación por el reciclaje de memoria de WSL, se aplicó una estimación algorítmica unificada en ambos experimentos: se determinó la memoria total sumando el peso de la estructura base (`std::vector`) y el espacio auxiliar dinámico demandado por cada algoritmo.

2. Resultados y Análisis

2.1. Ordenamiento de Arreglos

Para analizar los algoritmos de ordenamiento, nos vamos a enfocar en las gráficas del dominio D7, ya que muestran las mismas tendencias que el D1 pero dejan ver mejor cómo afecta la distribución inicial de los datos al rendimiento.

Revisando el **tiempo de ejecución** (Figura 1) con escala Log10, se nota claramente que Merge Sort (línea azul) es sumamente estable. Se mantiene en su cota $\mathcal{O}(n \log n)$ sin importar si el arreglo viene ordenado, aleatorio o invertido. En cambio, Quick Sort (línea roja) rinde muy bien con datos aleatorios, pero rinde pésimo y cae en su peor caso $\mathcal{O}(n^2)$ cuando el arreglo ya viene ordenado o invertido. Por este motivo, se optó por no correr las pruebas de $N = 10^7$ para esos casos, ya que iban a tardar más de 4 horas. Por último, sort (línea verde) soluciona esto gracias a que implementa *Introsort*, logrando los mismos tiempos de Quick Sort pero sin caerse en los peores escenarios.

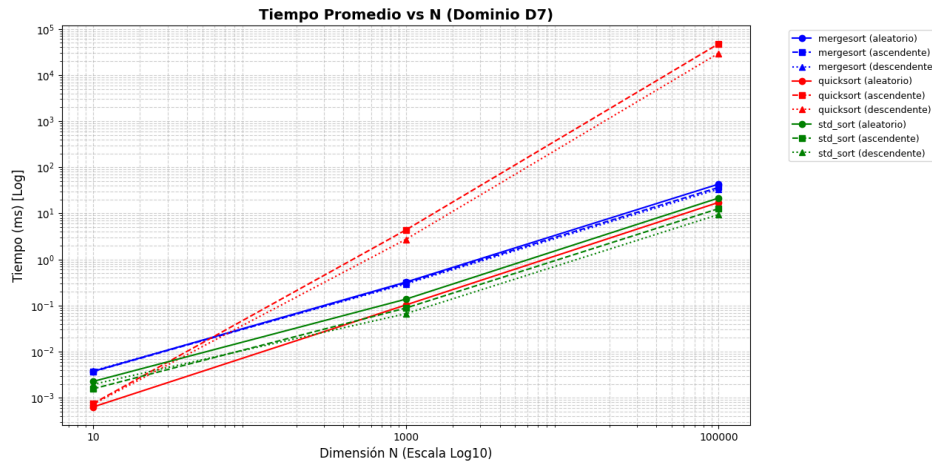


Figura 1: Tiempos promedio de ejecución en ordenamiento (Dominio D7). Se nota el salto cuadrático de Quick Sort con datos ordenados.

Respecto a la **memoria consumida** (Figura 2), la escala SymLog permite ver claramente que Quick Sort y sort operan de forma similar. Sus curvas son iguales porque solo usan la memoria del arreglo original más el pequeño costo de la pila de recursión ($\mathcal{O}(\log n)$). Por otro lado, Merge Sort necesita crear arreglos auxiliares para mezclar los datos, sumando un costo extra de $\mathcal{O}(n)$ que dispara su consumo frente al resto.

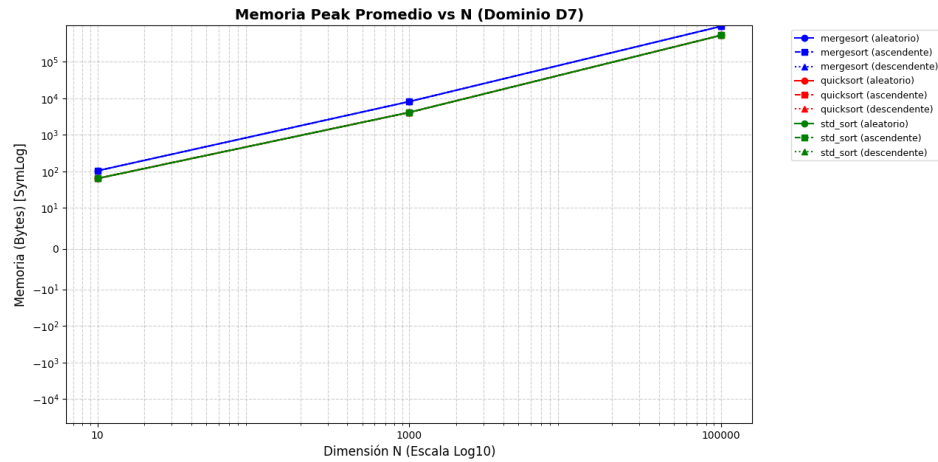


Figura 2: Consumo peak de memoria en ordenamiento (Dominio D7). La superposición de las curvas inferiores valida el enfoque in-place.

2.2. Multiplicación de Matrices

Este experimento demuestra cómo las constantes de la notación Big- $\mathcal{O}$ afectan en la práctica. Primero, vemos que las curvas de matrices densas, diagonales y dispersas son iguales; esto confirma que los algoritmos procesan todo de la misma forma al no tener optimizaciones para saltarse los ceros.

En el **tiempo de ejecución** (Figura 3), podemos ver aunque en teoría Strassen ($\mathcal{O}(n^{2.81})$) es más rápido que Naive ($\mathcal{O}(n^3)$), en la gráfica siempre se demora más. Por ejemplo, con $N = 1024$, Naive termina en $\approx 1,6$ ms, mientras que Strassen se termina demorando más de 160,000 ms. Esto ocurre porque el costo de la recursión en Strassen es mucho más pesado que correr los tres for simples de Naive para matrices de $N \leq 1024$.

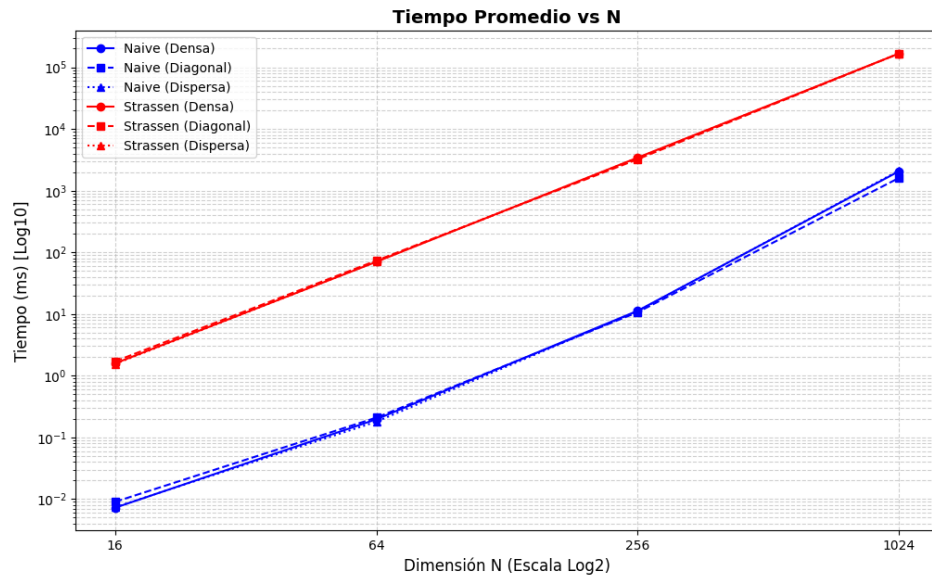


Figura 3: Tiempos promedio de ejecución en matrices. Escala Log-Log.

En cuanto a la **memoria consumida** (Figura 4), al sobrecargar new registramos toda la memoria solicitada en el Heap durante la multiplicación. El método Naive (línea azul) solo gasta lo necesario para guardar la matriz con el resultado final. Por el contrario, Strassen (línea roja) consume muchísima más memoria porque la recursión lo obliga a estar creando varias submatrices temporales de $N/2$ en cada paso, lo que dispara su consumo.

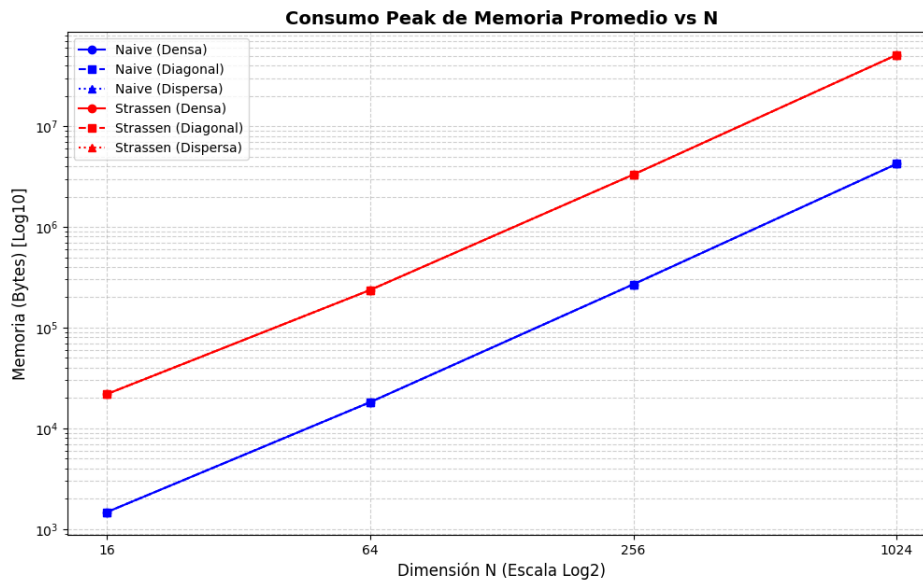


Figura 4: Memoria auxiliar dinámica peak en matrices. Escala Log-Log.

3. Conclusiones

La extensión máxima para esta sección es un párrafo breve (máximo 1/4 de página).

Escriba una conclusión breve que interprete los resultados más importantes. Cada afirmación debe estar respaldada por los datos presentados.